

RAG for a Production AI Assistant at Scale — Ideal Answer

System design mock #3 · 2026-09-24 · Topic from the daily briefing: retrieval infrastructure for a production AI assistant over millions of enterprise documents.

1. Requirements (ask these before drawing anything)

Functional - Ingest millions of enterprise documents (PDFs, Office docs, wiki pages, tickets, emails) and make them retrievable for grounded Q&A. - Answer natural-language questions with cited sources; stream responses. - Enforce per-user / per-group ACLs: a user must never see content they cannot access — not the text, not the existence of the document.

Non-functional (pinned for this round) - Read-heavy: 1,000 QPS peak, ~80/20 read/write on the document set. - Freshness: ~10k documents change per day; a new or updated document must be retrievable within 15 minutes. - Latency: p99 under 2 seconds for the full answer (streaming: first token much sooner). - Scale: millions of documents → tens of millions of chunks. - Correctness over cleverness: no hallucinated citations, no ACL leaks, measurable retrieval quality.

Out of scope (say it explicitly): training or fine-tuning the LLM, building the embedding model from scratch, multi-modal (images/video) retrieval.

2. Capacity estimation

- 5M documents × average 20 chunks = **100M chunk vectors**.
 - Embedding dim 768 (in-house model) × 4 bytes = ~3 KB per vector → **~300 GB of vectors** before index overhead. Fits on a modest sharded ANN cluster; comfortably fits object storage.
 - Query load: 1,000 QPS × 365 days — the expensive parts are embedding the query (~50 ms on GPU), ANN search (~50–150 ms), rerank, and LLM generation (dominant: 1–2 s streaming).
 - Ingestion: 10k doc changes/day ≈ 0.12 docs/sec average — trivially streamable; the 15-minute SLA is about pipeline latency, not throughput.
-

3. High-level architecture: two pipelines

Write path (ingestion): change-data-capture from source systems → parse/extract → chunk → embed (GPU batch workers) → write vectors to the ANN index + metadata to the document store.

Read path (query): authenticate → resolve user's ACLs → embed query → ACL-filtered ANN retrieval → rerank → build grounded prompt → stream LLM answer with citations → log for eval.

The two paths meet in two stores: a **metadata store** (DynamoDB: document registry, chunk hashes, ACLs, version timestamps) and a **vector index** (LanceDB-on-S3 style or a managed ANN service, sharded), with raw documents in S3.

4. Ingestion pipeline (write path)

1. **CDC / connectors:** each source (SharePoint, Confluence, file shares, ticket systems) gets a connector emitting create/update/delete events to a durable queue (Kafka/SQS). Deletes and permission changes are first-class events, not afterthoughts.
 2. **Parse:** a stateless worker pool extracts clean text (OCR for scans, table extraction for spreadsheets). Keep layout signals — headings and tables matter for chunk quality.
 3. **Chunk:** split into ~512-token chunks with ~64-token overlap. Store a **hash per chunk**. On document update, re-chunk and diff hashes: only new/changed chunks are re-embedded (delta embedding). Unchanged chunks keep their vectors — this is what makes the 15-minute SLA cheap.
 4. **Embed:** GPU batch workers run the in-house embedding model. Batching is the throughput lever: accumulate changed chunks for a few seconds, embed in one GPU batch.
 5. **Index write:** upsert vectors with metadata (doc_id, chunk_id, version, ACL labels) into the ANN index; write the document record, chunk hashes, and version timestamp to DynamoDB in the same transaction boundary (index write first, metadata second, with a reconciler for orphans).
 6. **Permission-change fast path:** ACL changes bypass re-embedding entirely — they only update metadata labels on the affected chunks. A permission revocation must take effect within the same 15-minute SLA.
-

5. Query path (read path) — the golden path

1. **AuthN/AuthZ:** authenticate the user; resolve their group memberships (AD/LDAP groups) once per session and cache with a short TTL (5 min) so a revocation propagates quickly.
2. **Query embedding:** embed the user query with the same model (GPU inference, ~50 ms).

3. **ACL-filtered ANN retrieval:** run the vector search **pre-filtered** by the user's ACL labels (metadata filter inside the index). Pre-filtering is the correct default: it guarantees the top-k are all visible to the user, and it never discloses the existence of restricted documents. Post-filtering is only defensible when the filter is extremely selective and you compensate with over-retrieval (fetch 5–10× k) — and even then, the UX must never reveal that hidden results exist.
4. **Rerank:** a cross-encoder reranker scores the top ~50 candidates down to the top 8–12. Vector search is high-recall; reranking buys precision, which is what citation quality depends on.
5. **Prompt construction:** assemble the context window — system instructions (grounding rules: "answer only from the provided sources; say you don't know otherwise"), the reranked chunks with doc IDs, and the user query. Budget the context window explicitly: e.g. 12 chunks × 400 tokens ≈ 5k tokens of context.
6. **Generate with citations:** stream the LLM response; require inline citations [doc_id] per claim. Constrain decoding so citations reference only chunk IDs present in the prompt.
7. **Judge check:** a lightweight LLM-as-judge verifies each cited claim is entailed by its cited chunk. On failure, either regenerate with a stricter prompt or fall back to an extractive answer (quote the chunks directly). This is the anti-hallucination gate.
8. **Log:** record query, retrieved chunk IDs, citations, judge verdict, latency breakdown, and user feedback signals for the eval loop.

p99 latency budget (2,000 ms)

- AuthN + ACL resolution (cached): ~10 ms
- Query embedding: ~50 ms
- ANN search (pre-filtered): ~100 ms
- Rerank 50 → 10: ~150 ms
- LLM time-to-first-token: ~300 ms, then streaming
- Judge check runs async / overlapped — not on the critical path for first token

6. ACL enforcement — the correct posture

- **Deny by default; no existence disclosure.** The system behaves as if restricted documents do not exist for that user. Never say "3 more results are hidden" — that leaks that matching restricted documents exist, which is itself a disclosure in a sensitive enterprise.
- **Pre-filter inside the index** using ACL labels stored as chunk metadata. Keep the label vocabulary small (group IDs, not per-user rows) so filters stay selective and the index stays fast.
- **Group membership cached with short TTL** (minutes, not hours) — revocation latency is a security property.

- **Audit every access:** log which chunks grounded which answer for which user. Compliance will ask.

The phrase to never use in an interview: "security by obscurity." Obscurity is not a control. The control is deny-by-default with no oracle — the system's responses must not let a user probe for the existence of documents they cannot see.

7. Freshness without breaking latency

- The 15-minute SLA is met by the **streaming ingestion pipeline**, not by the query path doing freshness checks. By the time a query arrives, the index already reflects the change.
 - **Cache versioning:** the semantic cache (below) keys entries by query embedding **plus the max chunk-version timestamp** of the corpus segments it touched — or simpler, a global "index version" watermark bumped by the ingestion pipeline. A query served from cache is valid iff its watermark matches the current one; on mismatch, re-retrieve. No per-query freshness RPC on the hot path.
 - **DynamoDB** holds the version watermarks and per-document timestamps; the query path reads the single global watermark (one cached read), not per-document checks.
-

8. Caching layers

1. **Exact query cache** (Redis): hash of normalized query + user ACL set → final answer. High hit rate on repeated enterprise questions ("what's the PTO policy?").
2. **Semantic cache:** store query embeddings with their answers in the vector index. On a new query, nearest-neighbor lookup above a high similarity threshold (e.g. 0.97) serves the cached answer. Invalidation via the watermark mechanism in §7 — when ingestion bumps the watermark, stale semantic entries stop matching.
3. **Embedding cache:** cache embeddings of frequent queries; trivial but saves GPU cycles at 1,000 QPS.

Cache keys always include the ACL set — a cached answer generated for one permission set must never serve a user with a different set.

9. Generation quality: citations and anti-hallucination

- **Grounded prompt contract:** system prompt instructs the model to answer only from provided chunks, cite every factual claim as `[doc_id]`, and say "I don't know" when the chunks don't cover the question. "I don't know" is a feature — it bounds hallucination.

- **Citation constraint:** post-process the output; any citation not in the retrieved set is stripped and the claim flagged.
- **LLM-as-judge:** an independent model checks claim $\Rightarrow$ chunk entailment. Judge disagreements route to the fallback: extractive summary (direct quotes) instead of abstractive generation.
- **No prompt injection from documents:** treat retrieved chunks as untrusted data, not instructions. Standard defense: clear instruction/data separation in the prompt template, and never let chunk text override system instructions.

10. Evaluation (how you know it's working)

- **Retrieval quality:** recall@k and nDCG on a golden set of question $\rightarrow$ relevant-chunk pairs, curated by domain experts and refreshed as the corpus changes. Track per-source-system (wiki vs tickets) to catch connector regressions.
- **Answer faithfulness:** judge-model entailment rate on sampled production traffic + human spot-checks. Alert on drops.
- **Citation precision:** % of citations that resolve to a chunk that actually supports the claim.
- **Freshness SLI:** % of document changes retrievable within 15 minutes.
- **Latency:** p50/p99 per stage (the budget in §5 becomes dashboards).
- **User signals:** explicit feedback (helpful/not), reformulation rate, and "I don't know" rate (too high = retrieval problem; too low = hallucination problem).

11. Failure modes

Failure	Mitigation
Embedding model down	Serve from semantic + exact caches; degrade to keyword (BM25) fallback index
ANN index shard down	Replicate shards; query remaining replicas, mark results partial
Ingestion lag (SLA breach)	Watermark exposes staleness; alert, and surface "index current as of HH:MM"
LLM provider outage	Extractive fallback: return top reranked chunks verbatim with citations
Poisoned/malicious document	Parse-stage sanitization; instruction/data separation; audit trail to source
ACL misconfiguration	Deny-by-default; periodic access-review reconciliation job diffing labels vs source ACLs

12. Session debrief (mock #3, 2026-09-24)

What went right - Clarifying questions before designing: rate, freshness, sensitivity, real-time constraints. Exactly right — do this every time. - Two-pipeline framing (ingestion vs query) with named stores (DynamoDB metadata, S3 documents, LanceDB-style vectors). - Chunk-hash deltas for cheap updates; ACL pre/post-filter tradeoff with UX reasoning; semantic cache with invalidation awareness.

Corrections to lock in - Never say "security by obscurity." The control is deny-by-default with no existence oracle. - The query path must be volunteered, not extracted: embed → pre-filtered ANN → rerank → grounded prompt → cited generation → judge check. - Freshness is a pipeline property (streaming ingestion + watermark), not a per-query check — keep freshness RPCs off the hot path. - Cache keys must include the user's ACL set.

Gaps to drill - p99 latency budget broken down by stage, stated unprompted. - Reranking: why vector search alone isn't enough for citation quality. - RAG eval: golden sets, faithfulness measurement, "I don't know" rate as a signal. - Generation details: context-window budgeting, citation constraints, prompt-injection defense.

Score: 6/10 — strong requirements and ingestion; query path, security phrasing, and generation depth need work.

13. The EM lens (say one of these to stand out)

- "I'd ship retrieval with eval before generation tuning — you can't improve what you can't measure, and the golden set is the team's contract with quality."
- "The ACL design is a product decision as much as a technical one: deny-by-default with no existence disclosure, and I'd get Security and Legal to sign off on that behavior explicitly."
- "Freshness SLA is an SLI with an alert, not a hope — the watermark makes staleness visible and the on-call knows exactly which pipeline stage breached it."